ICS 33 Fall 2026
Notes and Examples: Modules and Namespaces

Background

As you've likely discovered in your prior coursework, learning a programming language is a gradual process, rather than an immediate one. (This is especially true if Python is your first programming language, as it is for many of you.) Initially, you learn a couple of the language's constructs and build a simplified understanding of how they work individually and together, and how you might apply them to solve a small problem. Next, you learn a couple of additional constructs — perhaps motivated by a kind of problem that can't be solved in terms of the things you've already seen — and add them to your toolbox, seeing not only how you might use the new constructs in isolation, but in combination with the ones you've already learned.

Since you can't know all of the details of the language initially, your understanding is necessarily incomplete, yet you can still get your coursework done, partly because your instructors are being careful to assign problems that can be solved using only the techniques you're learning, but also because you've filled in the missing details in your mental model with reasonable assumptions. When you first encounter an `if` statement, you might see an example like this one:

```
>>> x = 20
>>> if x < 30:
...     print('Yes!')
... else:
...     print('No!')
...
    Yes!
```

An `if` statement is one that essentially allows you to decide "Should my program do this or not?" With `elif` or `else` clauses attached to it, its usefulness is extended, allowing you instead to decide "Should my program do this, that, or the other thing?" But, either way, the essential characteristic is there: What we're doing is answering a yes-or-no question and, based on that answer, doing one thing or another.

Python has a built-in type `bool` that naturally represents the answer to a yes-or-no question, so it's reasonable to assume that the conditional expression in an `if` statement would need to have the type `bool`. In fact, you can labor under that misconception for a long time while still getting programs written successfully. Curiously, though, when you dig deeper, you find something else, which you've likely seen by now, but that might have surprised you when you first saw it.

```
>>> name = 'Boo'
>>> if name:
...     print('Yes!')
... else:
...     print('No!')
...
    Yes!
>>> value = 0
>>> if value:
...     print('Yes!')
... else:
...     print('No!')
...
    No!
```

The course of action undertaken by an `if` statement is determined by what's called a *truth test*, where the conditional expression is evaluated for its *truthiness*. No matter what type of value is returned, that value will be considered either *truthy* or *falsy*. If it's truthy, the body of the `if` clause will execute; if it's falsy, we continue to the next clause instead.

When you first encounter an idea like this, it can reasonably lead to some questions, which are the kinds of questions I ask when I learn something new about a programming language.

- What determines truthiness? Under what circumstances is a non-`bool` value considered to be truthy? When is it falsy? (In other words, we first need a thorough understanding of the mechanics at work.)
- Under what circumstances would I want to use a non-`bool` expression in an `if` statement? (Now that we understand the mechanics, we should consider what we might use them to achieve.)
- How can I influence the result? I don't expect to be able to change how it works with, say, an integer, but suppose I write a new class called `Thingamabobber`. Under what circumstances would `Thingamabobber` objects be considered truthy or falsy? Does Python decide, or can I decide? (Now that we know how to use them, we should consider how to extend them to suit our needs.)

That's not to say that I'll always think about all three of these things immediately, but as I refine my understanding of a construct in a programming language, I'll eventually seek to understand all three of them. How does it work? What is it indended to be used for? How can I influence or build on its mechanics?

As you might have grown to expect, the third of these questions is where Python gives you a lot more flexibility than you'll find in a lot of other programming languages. A large percentage of the "inner workings" of Python are exposed not only in documentation, but in the language itself, which means that we have a wide variety of ways to influence them, so that we can build tools that (we hope) are clearer, easier to use, and more resistent to misuse. We can automate things that we might otherwise have to write repeatedly by hand, so that we can not only build them once, but also test them once. We can give users of our tools flexibility where it's useful, while limiting that flexibility where it's harmful.

So, as this course unfolds, you'll find this to be one of its central themes: Let's take a look at aspects of Python's mechanics in depth, so that we can understand how to use them, what to use them for, and how to extend them to suit our needs.

Taking a look within

One of the advantages the Python shell provides is its ability to let you inspect its current state, in many more ways than you might first think. For example, when you first start up a Python shell, you certainly expect to be able to use Python's built-in functions, operators, and so on.

```
>>> 11 + 7
    18
>>> list(range(5))
    [0, 1, 2, 3, 4]
```

But you can also find out what's available. The built-in function `dir` (short for "directory") provides a mechanism that you can think of as asking "What's available here?", with the goal of allowing a person using a Python shell to determine these things dynamically. So, if I start up a brand-new Python shell, what's available?

```
>>> dir()
    ['__builtins__', '__doc__', '__loader__', '__name__', '__package__', '__spec__']
```

Those are the identifiers that are defined in the current scope, which is the Python shell. Any of those identifiers can be evaluated and we'll see its value, which means we can also use the built-in `type` function to see its type.

```
>>> type(__name__)
    <class 'str'>
>>> __name__
    '__main__'
>>> type(__builtins__)
    <class 'module'>
>>> type(__builtins__.__dict__)
    <class 'dict'>
>>> len(__builtins__.__dict__)
    161
```

You've likely seen before that you can use the construct `if __name__ == '__main__'` to write code that runs only when a module is executed directly (as opposed to being imported). So, it shouldn't surprise us that there's always something called `__name__` that's available to us, no matter what its value (when evaluated in the Python shell) is `'__main__'`. That's what makes that mechanism work.

something called `__name__` that's available to us, nor that its value (when evaluated in the Python shell) is `__main__`. That's what makes that mechanism work.

The type of `__builtins__` suggests that it is itself a module, perhaps not unlike the ones that we write in `.py` files. So, why does it contain a dictionary named `__dict__`, and why does that dictionary have 161 keys in it? One way to find out is to take a look at what keys are in it.

```
>>> sorted(__builtins__.__dict__.keys())
    ['ArithmeticError', 'AssertionError', 'AttributeError', 'BaseException',
     'BaseExceptionGroup', 'BlockingIOError', 'BrokenPipeError', 'BufferError',
     'BytesWarning', 'ChildProcessError', 'ConnectionAbortedError', 'ConnectionError',
     'ConnectionRefusedError', 'ConnectionResetError', 'DeprecationWarning',
     'EOFError', 'Ellipsis', 'EncodingWarning', 'EnvironmentError', 'Exception',
     'ExceptionGroup', 'False', 'FileExistsError', 'FileNotFoundError',
     'FloatingPointError', 'FutureWarning', 'GeneratorExit', 'IOError', 'ImportError',
     'ImportWarning', 'IndentationError', 'IndexError', 'InterruptedError',
     'IsADirectoryError', 'KeyError', 'KeyboardInterrupt', 'LookupError',
     'MemoryError', 'ModuleNotFoundError', 'NameError', 'None', 'NotADirectoryError',
     'NotImplemented', 'NotImplementedError', 'OSError', 'OverflowError',
     'PendingDeprecationWarning', 'PermissionError', 'ProcessLookupError',
     'PythonFinalizationError', 'RecursionError', 'ReferenceError', 'ResourceWarning',
     'RuntimeError', 'RuntimeWarning', 'StopAsyncIteration', 'StopIteration',
     'SyntaxError', 'SyntaxWarning', 'SystemError', 'SystemExit', 'TabError',
     'TimeoutError', 'True', 'TypeError', 'UnboundLocalError', 'UnicodeDecodeError',
     'UnicodeEncodeError', 'UnicodeError', 'UnicodeTranslateError', 'UnicodeWarning',
     'UserWarning', 'ValueError', 'Warning', 'WindowsError', 'ZeroDivisionError', '_',
     '_IncompleteInputError', '__build_class__', '__debug__', '__doc__', '__import__',
     '__loader__', '__name__', '__package__', '__spec__', 'abs', 'aiter', 'all',
     'anext', 'any', 'ascii', 'bin', 'bool', 'breakpoint', 'bytearray', 'bytes',
     'callable', 'chr', 'classmethod', 'compile', 'complex', 'copyright', 'credits',
     'delattr', 'dict', 'dir', 'divmod', 'enumerate', 'eval', 'exec', 'execfile',
     'exit', 'filter', 'float', 'format', 'frozenset', 'getattr', 'globals',
     'hasattr', 'hash', 'help', 'hex', 'id', 'input', 'int', 'isinstance',
     'issubclass', 'iter', 'len', 'license', 'list', 'locals', 'map', 'max',
     'memoryview', 'min', 'next', 'object', 'oct', 'open', 'ord', 'pow', 'print',
     'property', 'quit', 'range', 'repr', 'reversed', 'round', 'runfile', 'set',
     'setattr', 'slice', 'sorted', 'staticmethod', 'str', 'sum', 'super', 'tuple',
     'type', 'vars', 'zip']
```

As someone who's written a fair amount of Python previously, many of those names will be familiar to you, even if many others will not. These are all names that are built into Python (i.e., things you can use without having to import any modules), though it turns out that you can also refer to those same names via the `__builtins__.__dict__` dictionary. (It's rarely the case you'd want to do so, but our current goal is to understand the mechanics, so it's worth seeing where our experiments lead us.)

```
>>> __builtins__.__dict__['list'](range(5))
    [0, 1, 2, 3, 4]
>>> __builtins__.__dict__['__name__']
    'builtins'
>>> __builtins__.__dict__['__doc__']
    "Built-in functions, exceptions, and other objects.\n\nNoteworthy: None is
    the `nil' object; Ellipsis represents `...' in slices."
>>> __builtins__.__dict__['False']
    False           # This is the actual value associated with the key 'False'.
>>> __builtins__.__dict__['False'] is False
    True            # They're the same object!
```

Now, if `__builtins__.__dict__` is really a Python dictionary, then this suggests that we can do more than just obtain values from it. Given a dictionary, we can also do these things (among others):

- Associate a value with a new key that isn't already in the dictionary.
- Change the value associated with a key that already exists.
- Remove a key and its associated value.

Can we do those kinds of things with the dictionary stored in `__builtins__.__dict__`, then? And if this dictionary represents the module's attributes, does that change what's considered "built into" Python?

```
>>> __builtins__.__dict__['booize'] = lambda x: f'Boo says {x}'
>>> booize('Hello')
    'Boo says Hello'    # Yep!  Now there's a built-in booize() function,
                        # which there always should have been!
>>> del __builtins__.__dict__['abs']
>>> abs(-3)
    Traceback (most recent call last):
      File "<python-input-1>", line 1, in <module>
        abs(3)
        ^^^
    NameError: name 'abs' is not defined
>>> __builtins__.__dict__['list'] = __builtins__.__dict__['set']
>>>
    # At this point, at least when I most recently tried it, all hell broke loose in the Python
    # shell after I then starting typing another expression, presumably because attempts to use
    # the 'list' type internally by the Python shell are now finding 'set' instead, with
    # catastrophic consequences!
```

So, all in all, it looks like we can make modifications to modules — even Python's `builtins` module! — on the fly. Don't worry; the next time you restart the Python shell, those changes will be lost, so we don't need to reinstall Python from scratch. But it's certainly an interesting thought that you might not have had about Python before. When they say Python is a "dynamic language," they aren't kidding!

Now, should we do those kinds of things with the `__builtins__` dictionary? Almost certainly not! But it's not a bad idea to stop and think about the limits of the programming language features you learn. Once you know how far they go, you'll be able to consider how you might use their flexibility for benefit instead of for causing trouble, and you may also find that there are ways to protect your programs against that sort of trouble being caused accidentally. (In a language that's as dynamic as Python, the meanings of all kinds of things can be changed, which means we have to take more care to be sure those meanings aren't changing in ways we don't want them to.)

Scopes, namespaces, and functions

Now that we've seen some of the mechanisms that allow Python to take an identifier and decide which object we've accessed, we should follow that trail further. If `dir()` gives us back a list of the names defined in the current scope, what happens to its result after we store a value in a new variable?

When we first start up a fresh Python shell in Visual Studio Code, this is its directory.

```
>>> dir()
    ['__builtins__', '__doc__', '__loader__', '__name__', '__package__', '__spec__']
```

What happens if we store the integer `3` into the variable `x`?

```
>>> x = 3
>>> dir()
    ['__builtins__', '__doc__', '__loader__', '__name__', '__package__', '__spec__', 'x']
                                                                        # ^^^ x is now in the directory.
```

We might expect, then, that deleting the variable subsequently would remove it from the directory, as well.

```
>>> del x
>>> dir()
    ['__builtins__', '__doc__', '__loader__', '__name__', '__package__', '__spec__']
                                                                    # ^^^ Yep!
```

What about defining functions? By what mechanism are they created?

```
>>> def square(n):
...     return n * n
...
>>> dir()
    ['__builtins__', '__doc__', '__loader__', '__name__', '__package__', '__spec__', 'square']
                                                                         # ^^^^^^ The mechanism is the same.
>>> square2 = lambda n: n * n
>>> dir()
    ['__builtins__', '__doc__', '__loader__', '__name__', '__package__', '__spec__', 'square', 'square2']
                                                                               # ^^^^^^^ Still the same.
>>> square(3)
    9
>>> square2(3)
    9
```

It turns out that `def` is (mostly) just a syntax for creating a function and storing it in a variable. There's at least one difference, though, between using `def` and, say, storing a function in a variable yourself: `def` also gives the function an explicit name.

```
>>> square
    <function square at 0x000001BAD90AD6C0>
>>> square2
    <function <lambda> at 0x000001BAD90AC8B0>
>>> square.__name__
    'square'
>>> square2.__name__
    '<lambda>'
```

Seeing this mechanism at work, you might expect to be able to give `square2` its own name, as well, by simply assigning to its `__name__` attribute, but this doesn't work as well as we might like.

```
>>> square2.__name__ = 'square2'
>>> square2.__name__
    'square2'
>>> square2
    <function <lambda> at 0x000001BAD90AC8B0>
            # ^^^^^^^^ No change here.
```

In practice, we want to be judicious about what kinds of behind-the-scenes modifications we make, because, as we see here, things aren't always what they seem. Python's documentation can help us to understand what kinds of modifications we can safely make (i.e., which ones are mechanisms that are meant to be overridden or extended) and which are on shakier ground. Some tweaks might depend on behavior that changes from one Python version to the next, or from one Python interpreter to another. That shouldn't stop us from experimenting and learning in the Python shell, but we want the techniques we use in our programs to be iron-clad, so we want to stick with the ones that are documented and intended to be used for customization.

Namespaces and shadowing

There's more to the story of how identifiers are *resolved* in Python, where "resolving an identifier" simply means "deciding what value we're talking about." You've likely seen some of these rules before, but let's be sure we all agree on how they work, and let's verify those details in a running Python program. To do that, we'll need to first understand a couple of additional functions built into Python: `globals()` and `locals()`.

```
>>> type(globals())
    <class 'dict'>
>>> sorted(globals().keys())
    ['__builtins__', '__doc__', '__loader__', '__name__', '__package__', '__spec__']
>>> type(locals())
    <class 'dict'>
>>> sorted(locals().keys())
    ['__builtins__', '__doc__', '__loader__', '__name__', '__package__', '__spec__']
```

This first attempt to use them hasn't shed a lot of light on the difference between them. They've both returned dictionaries containing the same keys we got back when we called `dir()` from the Python shell. So, this gives us an idea that they both have the goal of telling us what's available, but it's still unclear why both `globals()` and `locals()` would return the same keys.

Previously, you've likely learned that Python draws a distinction between *global variables* and *local variables*, but that this distinction is only meaningful within a function. Local variables are those that are accessible only within a function, while the global variables are the ones that are accessible throughout the module where that function resides. This strongly suggests that perhaps `globals()` and `locals()` will behave differently when run within a function. Let's find out.

```
>>> def something(n):
...     return globals(), locals()
...
>>> sg, sl = something(11)
>>> sorted(sg.keys())
    ['__builtins__', '__file__', '__loader__', '__name__', '__package__', '__spec__', 'sg', 'sl', 'something']
>>> sorted(sl.keys())
    ['n']
```

Now we're getting somewhere! `globals()` still returned what's globally accessible within the module — in this case, the Python shell — while `locals()` returned only the local variables from the function. Notably, `sg`, `sl`, and `something` are included among the keys stored in `sg`, which gives us an indication that `globals()` returns a reference to the globals, as opposed to a copy of it. (In truth, it returns the dictionary that stores the current module's attributes.)

Let's not forget that `sg` and `sl` are dictionaries, too, which means that we can see the values associated with each identifier, as well.

```
>>> sl['n']
```

```
    11
>>> sg['__name__']
    '__main__'
```

Suppose we execute the following Python module, `example.py`, which executes some code and prints some output describing some things about the program's state as it runs. What output would we expect to see?

example.py

```
print('In example module')
print(f'  globals: {sorted(globals().keys())}')
print(f'   locals: {sorted(locals().keys())}')


def foo(n):
    def bar(m):
        print('In bar function')
        print(f'  globals: {sorted(globals().keys())}')
        print(f'   locals: {sorted(locals().keys())}')
        return n + m

    print('In foo function')
    print(f'  globals: {sorted(globals().keys())}')
    print(f'   locals: {sorted(locals().keys())}')
    return bar(4)


print('Preparing to call foo function')
print(f'  globals: {sorted(globals().keys())}')
print(f'   locals: {sorted(locals().keys())}')
print('Calling foo function')
print(f'foo function returned {foo(2)}')
```

Here's the output I saw when I ran that module in the Python shell (with some extra space added for clarity).

```
    In example module
      globals: ['__builtins__', '__cached__', '__doc__', '__file__', '__loader__',
                '__name__', '__package__', '__spec__']
      locals: ['__builtins__', '__cached__', '__doc__', '__file__', '__loader__',
               '__name__', '__package__', '__spec__']
    Preparing to call foo function
      globals: ['__builtins__', '__cached__', '__doc__', '__file__', '__loader__',
                '__name__', '__package__', '__spec__', 'foo']
      locals: ['__builtins__', '__cached__', '__doc__', '__file__', '__loader__',
               '__name__', '__package__', '__spec__', 'foo']
    Calling foo function
    In foo function
      globals: ['__builtins__', '__cached__', '__doc__', '__file__', '__loader__',
                '__name__', '__package__', '__spec__', 'foo']
      locals: ['bar', 'n']
    In bar function
      globals: ['__builtins__', '__cached__', '__doc__', '__file__', '__loader__',
                '__name__', '__package__', '__spec__', 'foo']
      locals: ['m', 'n']
    foo function returned 6
```

So, what can we verify from all of this?

- When a module executes, its statements run in sequence. At the top, when we're printing the output labeled `In example module`, all we see in the output of both the `globals()` and `locals()` are the identifiers that are globally accessible. Importantly, neither `foo`, `bar`, `n`, nor `m` are listed, because none of them exists yet.
- After that, the `foo` function is created by the `def foo(n):` statement. Note that none of its output is printed, because we haven't called it (or the function `bar` inside of it) yet.
- Next, we see the output labeled `Preparing to call foo function`. Interestingly, one thing has changed: The `foo` function is now listed, because it now exists.
- At this point, we call the `foo` function, which tells us about its globals and locals. Knowing what we've learned in the past, it shouldn't surprise us that the globals are the same as what's listed at the module level; that's what globals are. The locals are a bit more interesting: `n` is `foo`'s parameter, while `bar` is also listed, because it's a local variable in `foo`, whose value is a function.
- Before returning, `foo` calls `bar`, which prints its own globals and locals. Still, the globals remain the same; they are what they are. The locals in `bar` are different, but mildly surprising in one way: Both `m` (`bar`'s parameter) and `n` (technically, a *nonlocal* variable from the enclosing scope) are listed.
- Finally, `bar` returns a value to `foo`, who, in turn, returns it back to the last line of our script, which prints its result. Its result is the sum of 2 and 4, which is 6.

How this relates to the LEGB rule

You may have seen before that Python resolves names within functions using a rule that is sometimes referred to as *LEGB*, which is an acronym standing for *Local*, *Enclosing*, *Global*, *Built-in*. When you specify an identifier in a Python function, this is how Python decides what you meant by it:

- L: If there's a local variable with that name, that's what you're referring to.
- E: Otherwise, if there's a variable in an enclosing scope with that name (e.g., when `bar` refers to one of `foo`'s variables in the example above), that's what you're referring to.
- G: Otherwise, if the identifier is defined globally (i.e., in the currently-executing module), that's what you're referring to.
- B: Otherwise, if the identifier is one of Python's built-ins, such as `list`, `str`, or `len`, that's what you're referring to.
- Otherwise, an exception will be raised, since the identifier has no accessible definition.

A natural consequence of the LEGB rule is that identifiers can *shadow* others, which is to say that you can define a local variable with the same name as a global variable in the same module. In the scope of that local variable, the local variable "wins" — though you might notice that a tool like `globals()` provides you with one possible workaround, albeit a heavy-handed one. In practice, your best bet is to limit the impact of this kind of shadowing wherever you can, by not attempting to rely on fancy techniques to work around it, but instead to respect the scopes introduced in your own designs. (This is one of many techniques to help a program make more sense to a human reader.) But it's handy to understand rules like these, because not understanding them can lead to not being able to understand one's own programs, especially as they change over time.

Modules and importation

Many things in Python can't be used unless we *import* the module in which they're defined. When we import a module, what happens? In what ways are those modules similar to the built-in module we saw before? In what ways are they different?

What happens, then, when we import a module from Python's standard library?

```
>>> import math
>>> math.sqrt(9)
    3.0                     # We can now call the math module's functions.
>>> type(math)
    <class 'module'>        # It's a module, just like builtins was.
>>> dir()
    ['__builtins__', '__doc__', '__loader__', '__name__', '__package__', '__spec__', 'math']
```

```
                                                                    # ^^^^ This is why we can say math.sqrt.
>>> sorted(math.__dict__.keys())
    ['__doc__', '__loader__', '__name__', '__package__', '__spec__', 'acos',
     'acosh', 'asin', 'asinh', 'atan', 'atan2', 'atanh', 'ceil', 'comb',
     'copysign', 'cos', 'cosh', 'degrees', 'dist', 'e', 'erf', 'erfc', 'exp',
     'expm1', 'fabs', 'factorial', 'floor', 'fmod', 'frexp', 'fsum', 'gamma',
     'gcd', 'hypot', 'inf', 'isclose', 'isfinite', 'isinf', 'isnan', 'isqrt',
     'lcm', 'ldexp', 'lgamma', 'log', 'log10', 'log1p', 'log2', 'modf', 'nan',
     'nextafter', 'perm', 'pi', 'pow', 'prod', 'radians', 'remainder', 'sin',
     'sinh', 'sqrt', 'tan', 'tanh', 'tau', 'trunc', 'ulp']
                              # Modules include a dictionary that contains their attributes.
>>> math.__dict__['sqrt'](9)
    3.0                       # We can access modules' attributes via that dictionary.
```

How is it different if we use the `from ... import` syntax to import a module instead? You've likely seen before that the reason for it is so we can access something in a module without specifying the name of the module. How do we expect this mechanism to work differently than what we already saw? Our intuition might lead us to a hypothesis:

- When we said `import math`, `math` was added to the namespace associated with the current scope, so we saw it appear in the result of `dir()` afterward.
- We might expect `from math import sqrt` to bring `sqrt` into `dir()` directly, then, so that we could subsequently say `sqrt(9)`.

An interesting open question is what happens to the rest of `math`? Does it get imported? Now that we know about directories and attribute dictionaries, we can answer our own question in the Python shell. Restarting the Python shell again, we could try the following experiment to get to the bottom of all of this.

```
>>> dir()
    ['__builtins__', '__doc__', '__loader__', '__name__', '__package__', '__spec__']
>>> from math import sqrt
>>> dir()
    ['__builtins__', '__doc__', '__loader__', '__name__', '__package__', '__spec__', 'sqrt']
```

We can see that `sqrt` is now listed in the top-level directory, so we would expect to be able to access it directly. Notably absent from the directory is `math`, so we might reasonably expect not to be able to access any of the rest of the `math` module, with or without qualification.

```
>>> sqrt(9)
    3.0
>>> math.pow(11, 7)          # Let's try something else from math.
    Traceback (most recent call last):
      File "<python-input-4>", line 1, in <module>
        math.pow(11, 7)
        ^^^^
    NameError: name 'math' is not defined. Did you forget to import 'math'?
>>> math.sqrt(9)             # How about math.sqrt?
    Traceback (most recent call last):
      File "<python-input-5>", line 1, in <module>
        math.sqrt(9)
        ^^^^
    NameError: name 'math' is not defined. Did you forget to import 'math'?
```

This also goes a long way toward explaining why indiscriminate use of the `from ... import *` syntax is so problematic. Think about what it does to the top-level directory! Starting again from a fresh Python shell, let's try it.

```
>>> len(dir())
    6
>>> from math import *
>>> from socket import *
>>> from pathlib import *
>>> len(dir())
    300      # That's a lot of top-level identifiers!  And each name can only
             # appear once in a given scope.  How confident are we that no
             # identifier is defined in two or more of those modules?  How
             # confident are we that the answer won't change in a future version
             # of Python?
```

Importation in scopes other than the global one

It's worth noting that while importation is most commonly done at the top level of a module, it doesn't have to be done there. Importation is a way to introduce identifiers into a scope, which can be done in local scopes (i.e., within functions), just as it can be done in global ones.

If we're curious how that might work, we now have the tools to find out with some experimentation. What output do we expect if we execute this module?

function_import.py

```
def hello():
    import math
    print('In hello')
    print(f'  globals: {sorted(globals().keys())}')
    print(f'   locals: {sorted(locals().keys())}')

hello()
print('Globally')
print(f'  globals: {sorted(globals().keys())}')
```

When I executed it in the Python shell, here's the output I got (with some additional spacing added for clarity).

```
    In hello
      globals: ['__builtins__', '__cached__', '__doc__', '__file__', '__loader__',
                '__name__', '__package__', '__spec__', 'hello']
       locals: ['math']
    Globally
      globals: ['__builtins__', '__cached__', '__doc__', '__file__', '__loader__',
                '__name__', '__package__', '__spec__', 'hello']
```

The most important thing we can see from this output is that `math` is only ever imported into `hello`'s local namespace. So, we would expect to be able to call a function such as `math.sqrt` from within `hello` (as long as we did so after we said `import math`), but we would not expect to be able to call `math.sqrt` anywhere else.

Importing the same module multiple times

Since we're aiming to get all the way to the bottom of the mechanisms at work during importation, there's one more wrinkle we should consider. What happens if we import the same module twice? What experiments should we run to understand the process more clearly?

We can begin with a fresh Python shell and use some techniques we've seen already.

```
>>> dir()
    ['__builtins__', '__doc__', '__loader__', '__name__', '__package__', '__spec__']
>>> import math
>>> dir()
    ['__builtins__', '__doc__', '__loader__', '__name__', '__package__', '__spec__',
     'math']
    # ^^^^ math appears now.
>>> import math
>>> dir()
    ['__builtins__', '__doc__', '__loader__', '__name__', '__package__', '__spec__',
     'math']
    # ^^^^ math still appears only once.
```

Why does `math` appear only once, even though we imported it twice? There are two reasons: one that's conceptual and another that's pragmatic.

- Conceptually, if we expect `import math` to mean "Make `math` available for use," then it's not unreasonable to expect that doing it twice means the same thing as doing it once. The `math` module, after all, is either available or it isn't.
- Pragmatically, we've seen that most of these mechanisms, behind the scenes, are driven by dictionaries. In a given dictionary, a particular key can only appear once (i.e., it can one have value associated with it). Consequently, we can't have two different meanings for the identifier `math`.

However, there are additional issues to consider here. What happens if we use `import` and `from ... import` on the same module? Starting with a fresh Python shell again, we can answer that question with similar techniques.

```
>>> dir()
    ['__builtins__', '__doc__', '__loader__', '__name__', '__package__', '__spec__']
>>> import math
>>> dir()
    ['__builtins__', '__doc__', '__loader__', '__name__', '__package__', '__spec__',
     'math']
    # ^^^^ There's math.
>>> from math import sqrt
>>> dir()
    ['__builtins__', '__doc__', '__loader__', '__name__', '__package__', '__spec__',
     'math', 'sqrt']
           # ^^^^ There's sqrt, listed separately.
>>> math.sqrt(9)
    3.0         # We can call math functions using math.XXX.
>>> math.log10(100)
    2.0         # All of math's functons are available that way.
>>> sqrt(9)
    3.0         # We can call math.sqrt without qualification.
>>> log10(100)
    Traceback (most recent call last):
      File "<python-input-9>", line 1, in <module>
        log10(100)
        ^^^^^
    NameError: name 'log10' is not defined
                # But we can't call any other math functions without qualification.
>>> math.sqrt is sqrt
    True        # The module is only loaded once.
                # We've just given different names to some of its attributes.
```

That last example — where we checked whether `math.sqrt` and `sqrt` lead to the same object — is more important than it looks, because it indicates something else about Python modules: In a running Python program, a module is only actually loaded the first time it's imported. Subsequently, it's available to be re-imported, but is not re-loaded separately every time. This has a performance benefit — since we aren't paying the cost of loading it repeatedly — but also an impact on the meaning of a program, especially in the (relatively rare) cases where importing a module has a side effect, such as printing something to the Python shell or performing expensive initialization that need only be done once. That will only happen the first time a module is imported, rather than every time.

Finally, we should consider the meaning of one more variant of importation in Python: `import ... as`. Starting, again, from a fresh Python shell, let's experiment with it.

```
>>> import math as m1
>>> import math as m2
        # At this point, we've imported the same module twice, but with different names.
>>> dir()
    ['__builtins__', '__doc__', '__loader__', '__name__', '__package__', '__spec__',
     'm1', 'm2']
   # ^^^^  ^^^^  There they are!
>>> m1.sqrt(9)
    3.0
>>> m2.sqrt(16)
    4.0
>>> math.sqrt(9)
    Traceback (most recent call last):
      File "<python-input-5>", line 1, in <module>
        math.sqrt(9)
        ^^^^
    NameError: name 'math' is not defined. Did you forget to import 'math'?
>>> m1.sqrt is m2.sqrt
    True        # The functions in the modules are the same.
>>> m1 is m2
    True        # The modules themselves are also the same.
```

None of these outcomes is in conflict with anything we've seen so far, so, at this point, we have our feet pretty firmly planted on the ground. Minor variants not explored here seem ever likelier to behave by the same rules we've seen, so the mechanisms by which modules are imported and used appear to be fully within our grasp now.